

PYTHON DİLİNE GİRİŞ

Python Dili Söz Dizim Kuralları

Python dilinde kaynak kod programcı tarafından yazılırken belli **söz dizim kullarına** (syntax) uyar. Bunlar aşağıda başlıklar halinde verilmiştir.

Kaynak kodumuz aşağıdaki karakterlerinden oluşabilir. Python 3.0 ve üzeri sürümlerde Unicode desteği yerleşik olarak bulunmaktadır. Python kaynak kodu için varsayılan kodlama UTF-8'dir. Ancak etik olarak aşağıdaki karakterler kullanılır;

1. Alfabedeki büyük ve küçük harfler (Unicode desteği bulunduğundan yerel karakterler de kullanılabilir):

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z

2. Rakamlar:

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

3. Özel Karakterler:

<	>	.	,	_	(	)	;	\$	:	%	[	]	#	?	'	&	{	}	"	^	!	*	/		-	\	~	+
---	---	---	---	---	---	---	---	----	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--	---	---	---	---

4. Metinde görünmeyen ancak metni biçimlendiren **beyaz boşluk** (white space) karakterleri;
 - a. Boşluk (space)
 - b. ↵ Yeni satır (new line)
 - c. → Sekme (tab)

Talimatlar ve Anahtar Kelimeler

Yüksek seviyeli bir dilde yazılmış bir **talimat** (statement), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, birkaç makine dili **emir kodunu** (instruction code) temsil edebilir. Talimatlar, yapılaması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (logical sequence).

Anahtar **kelimeler** (keyword), programlamada kullanılan ve Python yorumlayıcısı tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimeler** (reserved word) olup programı oluşturan **talimatlar** (statement) bu kelimeler kullanılarak yazılırlar.

```
>>> help()
Welcome to Python 3.13's help utility! If this is your first time using
Python, you should definitely check out the tutorial at
https://docs.python.org/3.13/tutorial/.

help> keywords

Here is a list of the Python keywords. Enter any keyword to get more help.

False      class      from       or
None       continue  global     pass
True       def       if         raise
and        del       import     return
as         elif      in         try
assert     else      is         while
async      except    lambda     with
await      finally  nonlocal   yield
break      for       not
```

Python dilinde;

- **Anahtar kelimeler**, küçük-büyük harf ayrımı yapıldığından her zaman **küçük harflerle yazılırlar**.
- **Talimatlar** (statement), **anahtar kelimeler içerecek şekilde yazılır**.

Normal olarak her satırda bir talimat yazılır. Ancak bazı durumlarda bir satırda birden fazla talimat yazılmak istenirse noktalı virgül (;) karakteri ile talimatlar ayrılır.

```
>>> toplam = 0.0
>>> sayaç = 1; adım = 10;
>>>
```

Açıklamalar

Özellikle dosyalarda saklanan **talimatları** (**statement**) yazarken bir açıklama yapmak istenildiğinde karma (#) karakteri kullanılır. Bu karakterden sonra satır sonuna kadar açıklama kabul edilir.

```
>>> indis=0 # bu değişken indis olarak kullanılacak
>>> toplam=0.0 # bu değişken işlemler toplamını tutacak
```

Birden fazla satırda açıklama üç adet tek tırnak veya çift tırnak karakterini art arda yazarak açıklamaya başlayabilir ve yine aynı şekilde açıklamayı bitirebiliriz.

```
>>> '''
...     İlhan Özkan
... Haziran 2025
... Ankara
... '''
'\nİlhan Özkan\nHaziran 2025\nAnkara\n' #Yukarıdakinin aynısıdır.
>>>
```

Değişkenler

Değişken Tanımlama

C/C++/C# ve Java gibi dillerin aksine Python dilinde tüm **değişkenler** (**variable**) aslında var olan bir **nesneye** (**object**) tanımlanan bir referanstır. Atama işlemi ile nesneye dönüştürülen **değişmezler** (**literal**) ile değişken tanımlanabilir. **Kimliklendirmede küçük büyük harf ayrımı yapılır. Ayrıca bir değişmeze atama yapamazsınız!**

```
Değişken-kimliği = değişmez
#aslında
Değişken-adı = herhangi-bir-nesne
```

Değişkenin hangi **veri tipinde** (**data type**) veri tutacağı dinamik olarak çalıştırma anında belirlenir. Burada belirlenen kimlikler aslında var olan nesnelere ilişkin referanslardır. Buradan anlaşılacağı üzere tüm değişmezler de nesne dönüştürülür ve sonrasında değişkene referansı verilir. Aşağıda örnekleri verilmiştir;

```
>>> tamsayı=1 #1 değişmezi bir int nesnesine dönüştürülür ve değişkene referans verilir
>>> print(tamsayı)
1
>>> büyüsayı=987654321987654321 #değişmez nesneye dönüştürülür ve değişkene referans verilir
>>> print(büyüsayı)
987654321987654321
>>> ondalık=3.14 #değişmez float nesnesine dönüştürülür ve değişkene referans verilir
>>> print(ondalık)
3.14
>>> avagadro=6.02214076e23
>>> print(avagadro)
6.02214076e+23
>>> karakter='A' #değişmez bir str nesnesine dönüştürülür ve değişkene referans verilir
>>> print(karakter)
A
>>> metin='İlhan Özkan'
>>> print (metin)
```

```

İlhan Özkan
>>> dogrumudegilmi=True
>>> print(dogrumudegilmi)
True
>>> tipibellidegil=None
>>> print(tipibellidegil)
None
>>> print(type(tamsayı))
<class 'int'>
>>> print(type(büyüksayı))
<class 'int'>
>>> print(type(ondalık))
<class 'float'>
>>> print(type(avagadro))
<class 'float'>
>>> print(type(karakter))
<class 'str'>
>>> print(type(metin))
<class 'str'>
>>> print(type(dogrumudegilmi))
<class 'bool'>
>>> print(type(tipibellidegil))
<class 'NoneType'>
>>>

```

Verilen örnekteki ilk satırda `tamsayı=1` talimatında `1` **değişmezi** (**literal**) için bellekte bir nesne oluşturulur ve tamsayı kimliği ile bu nesneye erişilebilir. Benzer şekilde `3.14`, `6.02214076e23`, `'A'` v e `True` değişmezleri için aynı durum geçerlidir.

Çoklu Değişken Tanımlama

Birden fazla değişkeni aralarında virgül kullanarak tek bir satırda tanımlayabilirsiniz;

```

>>> num1,num2,num3=10,20,30
>>> print(num1,num2,num3)
10 20 30
>>>

```

Birden fazla değişkeni aynı nesne ile tanımlayabilirsiniz;

```

>>> num1=num2=num3=100
>>> print(num1,num2,num3)
100 100 100
>>>

```

Değişken İsimlendirme Kuralları

Değişkenlere isim tanımlarken (kimlik verilirken) aşağıdaki **kurallara** (**naming convention**) uyulur;

1. Kimliklendirmede anahtar kelimeler kullanılamaz. Bu durumda `SyntaxError` hatası alırsınız.
2. Kimlikler küçük harf (a-z), büyük harf (A-Z), rakamlar (0-9) ve alt çizgi (_) içerebilir. UTF karakterleri de desteklendiğinden yerel alfabedeki karakterler de kullanılabilir.
3. Kimlikler bir rakamla başlayamaz. Örneğin, `1no` geçersiz bir değişken kimliğidir.
4. Kimlikler yalnızca rakamlardan oluşamaz. Örneğin, `888` geçersiz bir değişken kimliğidir.
5. Kimlikler altçizgi karakteri ile başlayabilir. Örneğin `_test` geçerli bir değişken kimliğidir.
6. Kimliklendirmede karakter sınırlaması yoktur. İstedığınız uzunlukta kimlik tanımlayabilirsiniz. Ancak, çok uzun bir değişken kimliği programcıyı yorar ve güvenilirliğinizi zedeler.
7. Kimliklendirmede **büyük küçük harf ayrımı yapılır**. Bu nedenle, `abc` ve `ABC` iki farklı değişken kimliğidir.

Kimliklendirmede, kodun okunaklılığı fazla olması için, etik olarak aşağıdaki üç desenden biri kullanılır;

- **Camel case:** İlk harf küçük harftir, ancak sonraki her kelimenin ilk harfi büyük harftir. Örneğin: öğrenciYaşı, birimFiyat
- **Pascal case:** Her kelimenin ilk harfi büyük harftir. ÖğrenciYaşı, BirimFiyat
- **Snake case:** Kelimeleri ayırmak için tek alt çizgi karakteri kullanılır. Örneğin öğrenci_yaşı, birim_fiyat

Değişkenlerin Bellek Adresleri

Bir değişkenin bellek adresi `id()` ile öğrenilebilir. Aşağıdaki örnekten de anlaşılacağı üzere değeri tamsayı bir olan değişmezden bir nesne imal edilip a değişkeni tanımlanmıştır. Aydında b değişkeni de a değişkeninin referans gösterdiği nesneyi gösterecek şekilde tanımlanmıştır. Her iki değişkenin de gösterdiği bellek adresi aynı olduğu gözden kaçırılmamalıdır.

```
>>> a=1
>>> b=a
>>> print(id(a),id(b))
140712917549992 140712917549992
>>>
```

C/C++/C# ve Java gibi dilleri **statik tip sistemine** (**static type system**) sahip olduklarından bu yapılamaz. Yani bir değişken kimliklendirildikten sonra veri tipi değiştirilemez. Çünkü veri tipine bağlı kimliklendirme yapılır. Aksine Python dilinde çalışma anında değişkenin işlediği veri tipi değiştirilebilir. Bu **dinamik veri tip sistemi** (**dynamic type system**) programcıya kolaylık sağlar;

```
>>> numara = 100
>>> print (numara)
100
>>> print(type(numara))
<class 'int'>
>>> numara = '0n'
>>> print (numara)
0n
>>> print(type(numara))
<class 'str'>
>>>
```

Mevcut Değişkenleri Görme

Yorumlayıcıda tanımlanan değişkenler tanımlandığı satır icra edildikten sonra hemen kaybolmazlar. Python **komut isteminde** (**command prompt**) aşağıdaki komut çalıştırılarak halihazırda tanımlı değişkenleri `dir()` komutuyla görebilirsiniz.

```
>>> a=10
>>> b=20
>>> print('toplam',a+b)
toplam 30
>>> print('çarpım',a*b)
çarpım 200
>>> dir()
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
'__name__', '__package__', '__spec__', 'a', 'b']
>>>
```

Tanımlı değişkenler listelenirken alt çizgi karakterleri arasında yer alan **ön tanımlı** (**default**) değişkenler de listelenir. Bu komutun dışında `locals()` ile yerel değişkenleri, `globals()` ile evrensel değişkenleri listeleyebilir ve değerlerini görebilirsiniz. İleride detayı verilecektir.

```
>>> locals()
{'__name__': '__main__', '__doc__': None, '__package__': '_pyrepl', '__loader__': None,
'__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>,
'__file__':
```

```
'C:\\Users\\ILHANOZKAN\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\_pyrepl\\__main__.py', '__cached__':
'C:\\Users\\ILHANOZKAN\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\_pyrepl\\__pycache__\\__main__.cpython-313.pyc', 'a': 10, 'b': 20}
>>> globals()
{'__name__': '__main__', '__doc__': None, '__package__': '_pyrepl', '__loader__': None,
 '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>,
 '__file__':
'C:\\Users\\ILHANOZKAN\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\_pyrepl\\__main__.py',
 '__cached__':
'C:\\Users\\ILHANOZKAN\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\_pyrepl\\__pycache__\\__main__.cpython-313.pyc', 'a': 10, 'b': 20}
>>>
```

Değişkenleri Silme

Halihazırda tanımlı değişkenleri silmek için `del` kullanılır.

```
>>> toplam=12
>>> print(toplam)
12
>>> del toplam
>>> print(toplam)
Traceback (most recent call last):
  File "<pyshell#28>", line 1, in <module>
    print(toplam)
NameError: name 'toplam' is not defined
>>>
```

Değişken Tiplerini Test Etme

Değişkenin veri tipi ön tanımlı `type()` fonksiyonu ile öğrenilir.

```
>>> a = 12
>>> print(type(a))
<class 'int'>
>>> b = '12'
>>> print(type(b))
<class 'str'>
>>> c = (1, 3.14)
>>> print(type(c))
<class 'tuple'>
>>>
```

Koşullu ifadelerde veri tipini `isinstance()` ile test etmek mümkündür. İleride anlatılacaktır.

Veri Tipleri

Dahili Veri Tipleri

bool

Doğru anlamında `True` veya yanlış anlamında `False` değer alabilen veri tipidir. `True` tamsayı olarak 1, `False` ise 0 sayısına karşılık gelir

```
>>> devammi=True
>>> print('Devam mı', devammi)
Devam mı True
>>> print(True+True)
2
>>> print(True*True)
1
>>>
```

int

Tamsayı değer alabilen veri tipidir. Python son sürümündeki bu veri tipi C dilindeki `long` veri tipine karşılık gelir.

```
>>> a = 1
>>> b = 300
>>> c = -123456789
>>> d = 398765432109876543
>>> print(a,b,c,d)
1 300 -123456789 398765432109876543
>>>
```

Değişmezler yukarıdaki gibi onluk sistemdeki tamsayılar olabileceği gibi **ikili** (**binary**), **sekizli** (**octal**) veya **on altılı** (**hexadecimal**) sayı tabanında olabilir. İkili sayılara ilişkin sabit örneği aşağıda verilmiştir.

```
>>> a=0b0101 # ikili sayı sabitinin başında 0b karakterleri bulunur.
>>> print(a," type a", type(a))
5 type a <class 'int'>
>>> b=int("0b1000",2)
>>> print(b," type b", type(b))
8 type b <class 'int'>
>>> c=bin(b)
>>> print(c,"type c:",type(c))
0b1000 type c: <class 'str'>
>>>
```

Sekizli sayı sisteminde bir sayıya ait değişmezin önünde sıfır ile birlikte küçük ya da büyük o harfi bulunur. Bir sayı ya da metinden sekizlik sistemde bir dizgi elde etmek istersek `oct()` fonksiyonunu kullanırız.

```
>>> a=0o10
>>> print(a,"type a",type(a))
8 type a <class 'int'>
>>> b=0020
>>> print(b,"type b",type(b))
16 type b <class 'int'>
>>> c=int('10',8)
>>> print(c,"type c",type(c))
8 type c <class 'int'>
>>> d=int('0o21',8)
>>> print(d,"type d",type(d))
17 type d <class 'int'>
>>> e=oct(16)
>>> print(e, "type e", type(e))
0o20 type e <class 'str'>
>>>
```

On altılık sistemde bir değişmezin önünde ise sıfır ile birlikte x harfi bulunur. Bir sayı ya da metinden on altılık sistemde bir dizgi elde etmek istersek `hex()` fonksiyonunu kullanırız.

```
>>> a=0x10
>>> print(a,'type a', type(a))
16 type a <class 'int'>
>>> b=0X20
>>> print(b,'type b', type(b))
32 type b <class 'int'>
>>> c=int('10',16)
>>> print(c,"type c",type(c))
16 type c <class 'int'>
>>> d=int('0x21',16)
>>> print(d,"type d",type(d))
```

```
33 type d <class 'int'>
>>> e=hex(10)
>>> print(e, "type e", type(e))
0xa type e <class 'str'>
>>>
```

float

Kayan noktalı sayıları tutan veri tipidir. Python son sürümündeki bu veri tipi C dilindeki `double` veri tipine karşılık gelir.

```
>>> a = 3.14
>>> b = 7.5e0
>>> c = 1e-1
>>> d = 0.5e8
>>> print(a,b,c,d)
3.14 7.5 0.1 50000000.0
>>>
```

Herhangi bir sayı ya da metni kayan noktalı sayıya çevirmek için `float()` fonksiyonu kullanılır. Matematikte sonsuzluk soyut bir kavramdır. Fiziksel olarak, sonsuz büyüklükteki sayılar hiçbir zaman herhangi bir miktardaki bellekte saklanamaz. Ancak bilgisayar donanım yapılandırmalarının çoğu için 10'un 400. kuvveti olan çok büyük bir sayı `Inf` ile gösterilir. `float()` fonksiyonu için argüman olarak "Infinity" kullanırsanız, `Inf` döndürür. Tanımlanmamış veya temsil edilemeyen herhangi bir değeri `Nan` (Not a Number) temsil eder.

```
>>> a=3.14E0
>>> print(a,type(a))
3.14 <class 'float'>
>>> b=float('Infinity')
>>> print(b,type(b))
inf <class 'float'>
>>> c=float('Nan')
>>> print(c,type(c))
nan <class 'float'>
>>>
```

complex

Karmaşık sayıları tutan veri tipidir. Sanal kısım `j` harfi ile biter.

```
>>> a = 3 + 5j
>>> b = -1.5 + 4.2j
>>> print (a,b)
(3+5j) (-1.5+4.2j)
```

str

Unicode karakterleri içeren metinleri veri olarak tutan `dizgilerdir` (`string`). `bytes` ise metin bayt dizisi karakterlerini içeren dizidir.

```
>>> metin='Uşagum Merhaba'
>>> batylar= u'Uşagum'
>>> print(metin)
Uşagum Merhaba
>>> dizgi = 'Hello World'
>>> print(dizgi[0]) #İlk karakteri konsola yazar. H
>>> print(dizgi[0:5]) #İlk 5 Karakteri konsola yazar. Hello
```

Python, dizgi `değişmezlerini` (`literal`) tanımlamak için tek tırnak (`'`), çift tırnak (`"`) ve üçlü tek veya çift tırnak (`"""`) işaretlerini, dizinin başında ve sonunda aynı tür tırnak işareti kullanıldığını sürece kabul eder.

```
>>> harf='a'
>>> kelime="sözcük"
>>> cümle="Türkçe güzel bir dildir."
```

```
>>> print(cümle)
Türkçe güzel bir dildir.
>>> paragraf="""Python dili çok esnek bir dildir.
Şöyleki C ve C++ dillerindeki dizki karakterleri gibi tanımlama yapabilirsiniz."""
>>> print(paragraf)
Python dili çok esnek bir dildir.
Şöyleki C ve C++ dillerindeki dizki karakterleri gibi tanımlama yapabilirsiniz.
```

`str` veri tipi Unicode karakterleri içerebilir, dolayısıyla tek, çift veya üçlü tırnaklı dize sözdizimi kullanılarak oluşturulan herhangi bir `dizgi (string)` Unicode olarak saklanır.

```
>>> string='ÜĞİŞÇÖİÜğışçöı'
>>> tb=string.encode('utf-8')
>>> print(tb)
b'\xc3\x9c\xc4\x9e\xc4\xb0\xc5\x9e\xc3\x87\xc3\x96\xc3\xbc\xc4\x9f\xc5\x9f\xc3\xa7\xc3\xb6\xc4\xb1'
>>> str=tb.decode('utf-8')
>>> print(str)
ÜĞİŞÇÖİÜğışçöı
```

Diziler ve Koleksiyonlar

Dizi ve koleksiyonlar `yinelenebilir` (`iterable`). Yani elemanları üzerinde `for` döngüsü ile gezinilebilir. Burada kısaca değinilen veri yapıları bir bölüm olarak detaylı incelenecektir.

str ve bytes

Bu veri tipleri karakterlerden oluşan `dizilerdir (array)`. *Dizi* başlığında anlatılacaktır.

```
>>> kelime="sözcük"
>>> print(kelime[0])
s
>>> print(kelime[1])
ö
```

tuple

`Demet (tuple)`, herhangi bir veri tipinde n adet değer bir araya gelerek oluşturduğu sıralı koleksiyondur. Elemanlar yuvarlak parantez içerisinde yazılarak ilk değer verilir.

```
>>> a = (1, 2, 3) # tamsayılardan oluşan demet
>>> print(a)
(1, 2, 3)
>>> b = (1, 'İlhan', 3.14, 'Özkan') # karışık veri tiplerinden oluşan demet
>>> print(b)
(1, 'İlhan', 3.14, 'Özkan')
>>> c = (1, 'ilhan', ('Özkan', 3.14)) # içinde bir başka demet olan demet
>>> print(c)
(1, 'ilhan', ('Özkan', 3.14))
>>> print(c[0])
1
>>> print(c[1])
ilhan
>>> print(c[2])
('Özkan', 3.14)
>>> d=(a,b,c) # başka demet elemanlar ile yeni demet tanımlama
>>> print(d)
((1, 2, 3), (1, 'İlhan', 3.14, 'Özkan'), (1, 'ilhan', ('Özkan', 3.14)))
>>> e=(100,) #tek elemanlı bir demet:
... #öncelik için kullanılan parantez ile karışmaması için eleman sonrasına bir virgül konulur
>>> print(e)
(100,)
```


İndisler (index) sıfırdan başlar ve her bir elemana köşeli parantez ([]) içerisinde indis kullanılarak erişilebilir. **Demet** (tuple) koleksiyonunda elemanlar **değiştirilemez** (immutable).

list

Listeler (list) herhangi bir veri tipindeki değerlerin sıralı bir koleksiyonudur. Elemanları **değiştirilebilir** (mutable). Elemanları köşeli parantez içerisinde yazılarak ilk değer verilir.

```
>>> a = [1, 2, 3]
>>> print(a)
[1, 2, 3]
>>> print(a[0]) #birinci eleman
1
>>> print(a[1]) #ikinci eleman
2
>>> print(a[2]) #Üçüncü eleman
3
>>> print(a[3]) #dördüncü eleman!!
Traceback (most recent call last):
  File "<pysHELL#5>", line 1, in <module>
    print(a[3])
IndexError: list index out of range
>>> print(a[-1]) #sondan birinci eleman
3
>>> print(a[-2]) #sondan ikinci eleman
2
>>> print(a[-3]) #sondan üçüncü eleman
1
>>> b = [1, 'ilhan', (3,5.0), ['özkan',12345]]
>>> print(b)
[1, 'ilhan', (3, 5.0), ['özkan', 12345]]
>>> b[0]=-1 # birinci eleman değiştiriliyor
>>> b[1]='özkan' # ikinci eleman değiştiriliyor
>>> b[2]= 3.14 # üçüncü eleman değiştiriliyor
>>> print(b)
[-1, 'özkan', 3.14, ['özkan', 12345]]
>>> c=[] #boş liste
>>> c.append(1) #listenin sonuna 1 sayısı eklendi.
>>> c.insert(0,100) #listenin sıfırıncı indisine 100 sayısı eklendi.
>>> print(c)
[100, 1]
>>> c.index(100) #100 sayısı listede hangi indiste?
0
>>> len(c) #c listesinin kaç elemanı var? Uzunluğu nedir?
2
>>> c.reverse() #c listesinin elemanlarını sondan başa yer değiştir. Tersini Al
>>> c
[100, 1]
>>> c[::-1] #c.reverse() ile aynı işi görür
[1, 100]
>>> c.pop() #varsayılan olarak son elemanı listeden çıkarır. Yada belirtilen indistekini.
1
>>> c
[100]
```

Dilim (slice) kavramı dizi, demet ve listenin elemanlarının bir veya birkaçını ifade etmek için kullanılır. Dilim, [başlangıç:bitiş:adım] şeklinde ifade edilir. Bu dilimin çıktısı, başlangıç indisinden **bitiş-1** indisine olan öğeleri içeren yeni bir listedir. **adım**, başlangıç ve **bitiş-1** arasında ile belirtilen aralıklar ile öğe seçimini ifade eder.

```
>>> liste = [1, 2, 3, 4]
>>> liste[1:]
```

```
[2, 3, 4]
>>> liste[:3]
[1, 2, 3]
>>> liste[::2]
[1, 3]
>>> liste[::-1]
[4, 3, 2, 1]
>>> liste[-1:0:-1]
[4, 3, 2]
>>> liste[4:9]
[]
>>> liste[1:10]
[2, 3, 4]
```

set

Küme (set), herhangi bir veri tipinde ve **tekil** (unique) elemanlardan oluşan sırasız bir koleksiyondur.

Bir nesne, ömrü boyunca asla değişmeyen bir **karma** (hash) değerine sahipse (bir `__hash__()` yöntemine ihtiyaç duyar) ve diğer nesnelerle karşılaştırılabiliriyorsa (bir `__eq__()` yöntemine ihtiyaç duyar) **karmalanabilir** (hashable). Eşitliği karşılaştıran karmalanabilir nesnelerin, aynı karma değerine sahip olması gerekir. Karma, bir nesneye eşsiz bir sayı olarak karşılık gelen değerlerdir.

```
>>> a = { 1, 2, 'a' }
>>> b = { 1, 1, 2, 2 } # bir değer bir kümede yalnızca 1 kez yer alabilir.
>>> print(b)
{1, 2}
```

dict

Sözlük (dict), **anahtar-değer çiftlerini** (key-value pair) tekil olarak tutan bir koleksiyondur. Anahtarlar **karmalanabilir** (hasable).

```
>>> a={ 1: 'Elma', 2: 'Armut', 3: 'Muz' }
>>> print(a)
{1: 'Elma', 2: 'Armut', 3: 'Muz'}
>>> b= { 'a': [1,2,3], 'b': (1,1), 'c': 'Özkan' }
>>> print(b)
{'a': [1, 2, 3], 'b': (1, 1), 'c': 'Özkan'}
```

Numaralandırma

Numaralandırma (enumeration), benzersiz, sabit bir değere bağlı bir dizi sembolik adlara sahip üyelerden oluşan değerlerdir. Bir numaralandırma içinde, üyeler kimliğe göre karşılaştırılabilir ve Numaralandırmanın kendisi **yenilenebilir** (iterable).

💡 Şimdiye kadar verilen örnekler Python **yorumlayıcısı** (interpreter) ile gerçekleştirilebilir. Bundan sonra verilecek örnekler bir Python dosyasına yazılmış gibi anlaşılacak **python dosya.py** şeklinde çalıştırılmalıdır.

Python 3.4 ve üzeri sürümlerde numaralandırma (enum) modülü standart kütüphanede bulunur, ayrı kurulum gerekmez.

```
#oynek.py
from enum import Enum
class Renkler(Enum):
    kırmızı = 1
    yeşil = 2
    mavi = 3
    sarı = 4
print(Renkler.kırmızı)
print(Renkler(1))
```

```
print(Renkler['kırmızı'])
print()
for renk in Renkler: #Numaralandırmalar yinelenebilir.
    print(renk)
```

Yukarıdaki örnek dosya `python ornek.py` şeklinde çalıştırılırsa aşağıdaki çıktı elde edilir.

```
Renkler.kırmızı
Renkler.kırmızı
Renkler.kırmızı

Renkler.kırmızı
Renkler.yeşil
Renkler.mavi
Renkler.sarı
```

Dahili Sabitler

Program içerisinde değerleri sabit olan nesneleri kullanırız. Bunlar;

- `True`: Dahili `bool` tipinin doğru değeri olan nesne.
- `False`: Dahili `bool` tipinin yanlış değeri olan nesne.
- `None`: Bir değer olmadığını belirtmek için kullanılan **tekil** (**unique**) bir nesnedir.
- `Ellipsis` veya `...`: İleride anlatılacaktır.
- `NotImplemented`: Bir yöntemin belirli argümanları desteklemediğini belirtmek için kullanılan tekil bir nesnedir.